

Département Mathématiques et Informatique
Filière(s) : II-BDCC

Chapitre 6 : Le Protocole http

Réalisé par :
KOUHAIL Aya

Supervisé par :
M. DAAIF Abdelaziz

Année Universitaire : 2025/2026

Table des matières

TP 1 : Exploration avec les DevTools	3
1.2 – Observer une requête simple.....	3
1.3 – Tester différentes méthodes	3
1.4 – Codes de statut observés	3
TP 2 : Maîtrise de cURL	4
2.1 – Différence entre -i et -v	4
2.2 – Requête POST avec données	4
2.3 – Headers personnalisés	4
2.4 – Suivre les redirections	4
2.5 – Exercice avancé : commande cURL complète	4
TP 3 : API REST avec JavaScript	6
3.1 à 3.4 – Opérations CRUD sur JSONPlaceholder.....	6
Exercice pratique : fetchWithRetry	6
TP 4 : Analyse des Headers de Sécurité	6
4.1 – Headers importants.....	7
4.2 – Analyse de 3 sites.....	7
TP 5 : Cache HTTP	8
5.1 – Headers de cache observés.....	8
5.2 – Requête conditionnelle avec ETag.....	8
5.3 – Configuration cache recommandée par type de fichier.....	8
Exercices Récapitulatifs	9
Exercice 2 : Questions théoriques.....	9
Conclusion.....	10

TP 1 : Exploration avec les DevTools

1.2 – Observer une requête simple

Résultats observés sur <https://httpbin.org/get> :

Code de statut : 200 OK — la requête a été traitée avec succès par le serveur.

Headers de requête envoyés :

- Host: httpbin.org
- User-Agent: Mozilla/5.0 (selon le navigateur utilisé)
- Accept: text/html, application/xhtml+xml, application/xml;q=0.9, */*;q=0.8
- Accept-Language: fr-FR,fr;q=0.9,en;q=0.8
- Accept-Encoding: gzip, deflate, br
- Connection: keep-alive

Content-Type de la réponse : application/json — le serveur renvoie les données au format JSON.

1.3 – Tester différentes méthodes

Requête GET exécutée via la console :

```
fetch('https://httpbin.org/get').then(r => r.json()).then(console.log);
```

Résultat : le serveur renvoie un objet JSON contenant les headers de la requête, l'IP d'origine, l'URL complète et les arguments (vides pour un GET simple).

Requête POST exécutée via la console :

```
fetch('https://httpbin.org/post', { method: 'POST',  
  headers: {'Content-Type': 'application/json'},  
  body: JSON.stringify({name: 'John', age: 30})  
}).then(r => r.json()).then(console.log);
```

Résultat : le serveur confirme la réception du corps JSON. L'objet json dans la réponse affiche {"name": "John", "age": 30}.

1.4 – Codes de statut observés

URL	Méthode	Code	Content-Type
httpbin.org/get	GET	200 OK	application/json
httpbin.org/post	POST	200 OK	application/json
httpbin.org/status/201	GET	201 Created	(vide)
httpbin.org/status/404	GET	404 Not Found	(vide)
httpbin.org/status/500	GET	500 Internal Server Error	(vide)
httpbin.org/redirect/3	GET	301 → 302 → ... → 200	application/json

TP 2 : Maîtrise de cURL

2.1 – Différence entre -i et -v

-i (--include) : Affiche uniquement les headers HTTP de la réponse suivis du corps. Utile pour vérifier rapidement les headers sans surplus d'informations.

-v (--verbose) : Affiche tout le dialogue HTTP : connexion TLS, headers envoyés (>), headers reçus (<), et corps. Indispensable pour déboguer les problèmes de requêtes.

2.2 – Requête POST avec données

Form data :

```
curl -X POST -d "name=John&email=john@example.com" https://httpbin.org/post
```

Résultat : le serveur reçoit les données dans le champ form et le Content-Type est automatiquement application/x-www-form-urlencoded.

JSON :

```
curl -X POST \
-H "Content-Type: application/json" \
-d '{"name": "John", "email": "john@example.com"}' \
https://httpbin.org/post
```

Résultat : les données sont reçues dans le champ json (non dans form), car le Content-Type est explicitement défini comme application/json.

2.3 – Headers personnalisés

```
curl -H "Authorization: Bearer mon-token-secret" \
-H "Accept: application/json" \
https://httpbin.org/headers
```

Résultat : l'objet headers dans la réponse JSON confirme que les deux headers ont bien été transmis au serveur.

2.4 – Suivre les redirections

Sans -L : cURL s'arrête à la première réponse 301/302 et affiche le header Location sans suivre la redirection.

Avec -L : cURL suit automatiquement toutes les redirections jusqu'à la réponse finale (200 OK). Dans l'exemple /redirect/3, il effectue 3 redirections avant d'obtenir la réponse définitive.

2.5 – Exercice avancé : commande cURL complète

Commande qui répond à tous les critères demandés :

```
curl -X POST \
-H "Content-Type: application/json" \
-H "X-Custom-Header: MonHeader" \
-d '{"action": "test", "value": 42}' \
-i \
```

`https://httpbin.org/post`

Explication : -X POST définit la méthode, -H ajoute les headers, -d envoie le corps JSON, et -i affiche les headers de réponse.

TP 3 : API REST avec JavaScript

3.1 à 3.4 – Opérations CRUD sur JSONPlaceholder

Les fonctions GET, POST, PUT et DELETE ont été testées avec succès sur <https://jsonplaceholder.typicode.com>.

Exercice pratique : fetchWithRetry

Fonction complète avec gestion des erreurs 5xx et mécanisme de réessai :

```
async function fetchWithRetry(url, options = {}, maxRetries = 3) {
  let lastError;
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      const response = await fetch(url, options);
      // Si erreur 5xx, on réessaie
      if (response.status >= 500 && response.status < 600) {
        lastError = new Error(`Erreur serveur : HTTP ${response.status}`);
        if (attempt < maxRetries) {
          console.warn(`Tentative ${attempt} échouée. Réessai dans 1s...`);
          await new Promise(resolve => setTimeout(resolve, 1000));
          continue;
        }
      } else {
        return response; // Succès ou erreur 4xx (pas de réessai)
      }
    } catch (networkError) {
      lastError = networkError;
      if (attempt < maxRetries) {
        await new Promise(resolve => setTimeout(resolve, 1000));
      }
    }
  }
  throw lastError;
}
```

Explication de la logique :

- La boucle tente la requête jusqu'à maxRetries fois.
- Les erreurs 5xx (serveur) déclenchent un réessai avec un délai d'1 seconde via setTimeout.
- Les erreurs 4xx (client) ne sont pas réessayées car elles indiquent une erreur de la requête elle-même.
- Les erreurs réseau (pas de connexion) sont également gérées dans le catch.
- Si toutes les tentatives échouent, l'erreur est propagée avec throw.

TP 4 : Analyse des Headers de Sécurité

4.1 – Headers importants

Header	But	Valeur recommandée
Strict-Transport-Security	Forcer HTTPS	max-age=31536000; includeSubDomains
X-Frame-Options	Anti-clickjacking	DENY ou SAMEORIGIN
X-Content-Type-Options	Anti-MIME sniffing	nosniff
Content-Security-Policy	Sources autorisées	default-src 'self'
Referrer-Policy	Contrôle du referrer	strict-origin-when-cross-origin

4.2 – Analyse de 3 sites

Site	HSTS	X-Frame	CSP	Note
github.com	Oui ✓	DENY ✓	Oui (stricte) ✓	A+
google.com	Oui ✓	SAMEORIGIN ✓	Partielle	A
example.com	Non ✗	Absent ✗	Absent ✗	F

Observations :

- github.com est l'exemple de référence : tous les headers de sécurité sont présents et correctement configurés.
- google.com a une CSP partielle car il utilise de nombreux sous-domaines et services tiers.
- Les sites sans HSTS sont vulnérables aux attaques de type downgrade HTTPS.

TP 5 : Cache HTTP

5.1 – Headers de cache observés

Commande : `curl -i https://httpbin.org/cache/60`

Headers de cache retournés par le serveur :

- Cache-Control: public, max-age=60 — la ressource peut être mise en cache pendant 60 secondes.
- ETag: "valeur-unique" — identifiant de version de la ressource pour la validation conditionnelle.
- Last-Modified: [date] — date de dernière modification pour les requêtes If-Modified-Since.

5.2 – Requête conditionnelle avec ETag

```
# Étape 1 : obtenir l'ETag
curl -i https://httpbin.org/etag/test123

# Étape 2 : requête conditionnelle
curl -i -H "If-None-Match: test123" https://httpbin.org/etag/test123
```

Résultat attendu : 304 Not Modified — le serveur confirme que la ressource n'a pas changé. Le navigateur peut utiliser sa version en cache, ce qui économise la bande passante.

5.3 – Configuration cache recommandée par type de fichier

Type de fichier	Cache-Control recommandé	Raison
Images (logo, icônes)	public, max-age=31536000, immutable	Fichiers stables, rarement modifiés
CSS / JS (avec hash)	public, max-age=31536000, immutable	Versionnés par hash de contenu
HTML	no-cache	Doit toujours être revalidé
API JSON dynamique	no-store	Données personnalisées, jamais en cache
Fonts	public, max-age=31536000	Fichiers statiques stables

Exercices Récapitulatifs

Exercice 2 : Questions théoriques

1. Différence entre no-cache et no-store

no-cache : La ressource PEUT être stockée en cache, mais le navigateur doit TOUJOURS la revalider auprès du serveur avant de l'utiliser (via ETag ou Last-Modified). Si le serveur confirme que la ressource n'a pas changé (304 Not Modified), la version en cache est utilisée.

no-store : La ressource NE DOIT JAMAIS être stockée en cache, ni par le navigateur ni par les proxies intermédiaires. Chaque requête télécharge la ressource depuis le serveur. Utilisé pour les données sensibles (fiches bancaires, données médicales).

2. Pourquoi POST n'est-il pas idempotent ?

Une opération est idempotente si l'appliquer plusieurs fois produit le même résultat que l'appliquer une seule fois. POST n'est pas idempotent car chaque appel crée une nouvelle ressource distincte. Par exemple, envoyer deux fois POST /articles crée deux articles séparés avec deux IDs différents. En revanche, PUT /articles/1 appliqué plusieurs fois modifie toujours le même article sans créer de doublons, ce qui le rend idempotent.

3. Que se passe-t-il avec un code 301 ?

Un code 301 (Moved Permanently) indique que la ressource a été déplacée définitivement vers l'URL fournie dans le header Location. Les conséquences sont :

- Le navigateur redirige automatiquement la requête vers la nouvelle URL.
- Les moteurs de recherche (SEO) transfèrent le PageRank vers la nouvelle URL.
- Le navigateur met en cache la redirection et l'utilisera directement sans contacter le serveur lors des visites suivantes.
- La méthode HTTP peut être changée : un POST peut devenir un GET après une redirection 301 (contrairement au 308 qui préserve la méthode).

4. À quoi sert le header Origin ?

Le header Origin indique l'origine (protocole + domaine + port) de la page qui effectue la requête. Il est envoyé automatiquement par le navigateur dans les requêtes cross-origin (CORS) et les requêtes POST. Le serveur l'utilise pour décider si la requête doit être autorisée via les headers Access-Control-Allow-Origin. Contrairement au header Referer qui contient l'URL complète, Origin ne contient que l'origine sans le chemin, ce qui préserve mieux la confidentialité.

5. Pourquoi utiliser HttpOnly sur les cookies de session ?

L'attribut HttpOnly empêche tout accès au cookie via JavaScript (document.cookie). Cela protège contre les attaques XSS (Cross-Site Scripting) : même si un attaquant injecte du code JavaScript malveillant dans la page, il ne peut pas lire ni voler le cookie de session. Sans HttpOnly, un simple script comme document.cookie suffirait à exfiltrer le token d'authentification vers un serveur distant. Il est recommandé de combiner HttpOnly avec l'attribut Secure (cookie transmis uniquement en HTTPS) et SameSite=Strict (protection contre le CSRF).

Conclusion

Ces travaux pratiques ont permis d'explorer en profondeur le protocole HTTP à travers plusieurs angles :

- L'analyse des requêtes/réponses avec les DevTools du navigateur et cURL en ligne de commande.
- L'utilisation de l'API Fetch en JavaScript pour effectuer des opérations CRUD sur une API REST.
- La compréhension des headers de sécurité et leur importance pour protéger les applications web.
- Le fonctionnement du cache HTTP et les stratégies d'optimisation des performances.

La maîtrise de ces concepts est fondamentale pour le développement web moderne, que ce soit côté client ou serveur, et constitue la base de la communication entre toutes les applications web.